

- 1. Título e Introducción
- 2. Planteamiento y Datos
- 3. Objetivos y Justificación
- 4. Metodología y Ejecución
 - Fase 3: Solución Heurística
 - Fase 4: Modelo MLP
 - Fase 5: Evaluación
- 5. Conclusión

Proyecto Integrador

Optimización de Asignación de Turnos mediante Inteligencia Artificial y Redes Neuronales

[cite: 3]

Introducción

El scheduling (planificación de tareas) es un problema fundamental en múltiples áreas como la administración, la industria y los sistemas computacionales[cite: 5]. Consiste en asignar recursos a tareas de manera eficiente, considerando restricciones y objetivos específicos[cite: 6].

En este proyecto abordamos este problema integrando técnicas de aprendizaje automático, particularmente redes neuronales multicapa (MLP), para proponer soluciones adaptativas superiores a los métodos tradicionales[cite: 7].

Planteamiento del Problema y Datos

El problema exige asignar un conjunto de empleados a una serie de turnos semanales[cite: 9]. Se debe cumplir con la cobertura total de turnos, no exceder las horas máximas por empleado, respetar disponibilidades y distribuir el trabajo equilibradamente[cite: 17, 18, 19, 20, 21]. Las

Scheduling MVP

- 1. Título e Introducción
- 2. Planteamiento y Datos
- 3. Objetivos y Justificación
- 4. Metodología y Ejecución
 - Fase 3: Solución Heurística
 - Fase 4: Modelo MLP
 - Fase 5: Evaluación
- 5. Conclusión

restricciones asumen que cada turno equivale a 5 horas y un empleado toma máximo un turno diario[cite: 28, 29].

Mapecto de Datos (JSON)

Los datos teóricos [cite: 22-26] fueron estructurados en diccionarios JSON para su ingestión en los modelos de Python.

empleados.json

```
[
  {
    "id": "E1", "nombre": "Ana", "horas_max_semana": 40,
    "disponibilidad": { "Lunes": ["Mañana"], "Martes": ["Mañana"]... }
  },
  {
    "id": "E2", "nombre": "Luis", "horas_max_semana": 30,
    "disponibilidad": { "Lunes": ["Tarde"], "Martes": ["Tarde"]... }
  }...
]
```

turnos.json

```
[
  {"id": "T1", "dia": "Lunes", "hora": "Mañana"},
  {"id": "T2", "dia": "Lunes", "hora": "Tarde"},
  ...
]
```

Objetivos y Justificación

Objetivo General: Desarrollar una solución basada en IA para optimizar la asignación de turnos[cite: 33].

- Modelar el problema identificando variables y restricciones[cite: 35, 36].
- Implementar una solución base (Heurística)[cite: 36].
- Diseñar un modelo de red neuronal (MLP) y comparar resultados[cite: 37].

- 1. Título e Introducción
- 2. Planteamiento y Datos
- 3. Objetivos y Justificación
- 4. Metodología y Ejecución
 - Fase 3: Solución Heurística
 - Fase 4: Modelo MLP
 - Fase 5: Evaluación
- 5. Conclusión

El uso de IA en scheduling permite mejorar la eficiencia operativa, adaptarse a entornos dinámicos y reducir costos operacionales aprendiendo de datos históricos[cite: 39, 40, 41].

Metodología y Ejecución del Código

El proyecto se dividió en fases lógicas [cite: 46-63]. Las Fases 1 y 2 correspondieron al análisis del escenario y definición de variables [cite: 46-51]. La ejecución técnica abarca las Fases 3 a 5.

Fase 3: Solución Base (Heurística) [cite: 52, 53]

Se desarrolló `heuristica.py` utilizando un enfoque Greedy. La estrategia filtra candidatos válidos según disponibilidad y restricciones, priorizando a aquellos con menos horas asignadas para mantener el equilibrio.

asignacion_heuristica.json (Resultado)

```
{
  "metodo": "heuristica_greedy",
  "resumen": {
    "turnos_cubiertos": 4,
    "turnos_total": 8,
    "horas_por_empleado": {
      "E1": 15, "E2": 15, "E3": 5, "E4": 15, "E5": 10
    }
  }
}
```

- 1. Título e Introducción
- 2. Planteamiento y Datos
- 3. Objetivos y Justificación
- 4. Metodología y Ejecución
 - Fase 3: Solución Heurística
 - Fase 4: Modelo MLP
 - Fase 5: Evaluación
- 5. Conclusión

Fase 4: Implementación de IA (MLP) [cite: 54-57]

Se diseñó una Red Neuronal Multicapa en `modelo_mlp.py` utilizando Keras/TensorFlow[cite: 67]. La arquitectura consta de una capa de entrada (10 features), tres capas ocultas densas (64, 32, 16 neuronas con activación ReLU) y una salida sigmoide para predecir la probabilidad de asignación.

Arquitectura (`modelo_mlp.py`)

```
modelo = keras.Sequential([
    layers.Input(shape=(X.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
modelo.compile(optimizer='adam',
               loss='binary_crossentropy',
               metrics=['accuracy'])
```

Métricas (`historial_entrenamiento.json`)

```
{
  "loss_final": 0.0959,
  "accuracy_final": 0.9421,
  "val_loss_final": 0.0971,
  "val_accuracy_final": 0.9390,
  "epochs": 50
}
```

El modelo generó el archivo `asignacion_mlp.json`, logrando probabilidades de predicción que posteriormente pasan por un filtro estricto de restricciones operativas.

- 1. Título e Introducción
- 2. Planteamiento y Datos
- 3. Objetivos y Justificación
- 4. Metodología y Ejecución
 - Fase 3: Solución Heurística
 - Fase 4: Modelo MLP
 - Fase 5: Evaluación
- 5. Conclusión

Fase 5: Evaluación y Comparación [cite: 58-60]

El script de evaluación generó `comparacion.json`. Al medir la cobertura, equilibrio y utilización, tanto el modelo heurístico como la red neuronal arrojaron resultados idénticos bajo el set de datos actual.

```
{
  "ganadores": {
    "cobertura": "Empate",
    "equilibrio": "Empate",
    "utilizacion": "Empate"
  },
  "metricas": {
    "heuristica": { "cobertura_pct": 50.0, "horas_promedio": 12.0, "horas_std": 4.0 },
    "mlp": { "cobertura_pct": 50.0, "horas_promedio": 12.0, "horas_std": 4.0 }
  }
}
```

Nota de escalabilidad: Aunque hay empate en este conjunto de datos pequeño, la ventaja del MLP radica en su capacidad de generalizar variables no lineales (como ausentismo histórico o picos de demanda) cuando se despliega en producción con un pipeline RAG o un orquestador más complejo.

Conclusiones [cite: 61, 84]

Este proyecto demuestra la viabilidad técnica de trasladar un problema teórico de scheduling [cite: 85] a un entorno de software estructurado. Se cumplió el objetivo de integrar IA moderna y redes neuronales multicapa frente a lógicas tradicionales[cite: 85].

Para escalar a un entorno de producción, la base actual permite abstraer la lógica de `predecir_asignacion` como un microservicio (ej. en App Engine o Vertex AI), fortaleciendo la

Scheduling MVP

- 1. Título e Introducción
- 2. Planteamiento y Datos
- 3. Objetivos y Justificación
- 4. Metodología y Ejecución
 - Fase 3: Solución Heurística
 - Fase 4: Modelo MLP
 - Fase 5: Evaluación
- 5. Conclusión

infraestructura tecnológica de la empresa y la capacidad de resolver problemas complejos del mundo real[cite: 86].